

통합 디버깅과 E2E 테스트

Playwright MCP와 자율 디버깅 3원칙 — 시나리오 검증 루프

— 본 자료의 구성

01 통합 테스트와 E2E

02 Playwright MCP와 자율 디버깅 3원칙

03 docs/prd.md 기반 시나리오 검증 루프

— SECTION 1

통합 테스트와 E2E

- 01 3가지 테스트 단계를 구분한다
- 02 시나리오 통과가 동작의 정의임을 인식한다
- 03 화면별 동작과 시나리오 통과의 차이를 설명한다

자료 6 종료 상태와 본 단계의 본질

자료 6 끝 손에 있는 것

- 동작하는 화면 2개 (카피 입력 + 카피 결과)
- prd-reviewer + git-committer 두 sub-agent
- PostToolUse + Stop 두 hook
- 병합된 PR 9개

미확인 사항

- 화면 1 → 화면 2 시나리오가 끝까지 통과하는가?
- 사용자가 입력부터 결과까지 한 흐름으로 완수하는가?

화면별 동작은 확인됐지만 **시나리오 통과**는 별개의 검증. 본 단계가 그 빈자리를 채운다.

첫 단정

동작한다의 정의는
테스트가 통과한다이다

— 테스트 단계

3가지 테스트 단계

단계	검증 대상	본 학습 합격선
단위 테스트	한 함수	보조
통합 테스트	여러 모듈 연결	보조
E2E 테스트	사용자 시나리오	핵심

단위 테스트 (Unit Test)

한 함수가 올바른 값을 반환하는지 검증.

"이 함수에 입력 A를 주면 출력 B가 나오는가"

가장 작은 단위. 빠르게 실행. 하지만 전체 흐름은 알 수 없다.

통합 테스트 (Integration Test)

여러 모듈이 합쳐졌을 때 데이터 흐름이 정상인지 검증.

"이 화면에서 저 화면으로 데이터가 잘 넘어가는가"

단위보다 큰 범위. 모듈 간 연결 확인.

E2E 테스트 (End-to-End Test)

사용자가 처음부터 끝까지 시나리오를 완수할 수 있는지 검증.

시드 6번 예시:

셀러가 카피 입력 화면에 상품 사진과 키워드 5개를 입력하고
'카피 생성' 버튼을 클릭하면 결과 화면에
차별화 카피 3개가 표시된다

가장 큰 범위. 실제 사용자 경험 시뮬레이션.

시나리오 통과 vs 화면 땀

"화면이 땀으면 동작하는 것 아닌가?"

화면이 떠도 시나리오 통과는 별개.

화면 땀	시나리오 통과
카피 입력 화면 렌더링 OK	입력 → 결과 화면 라우팅 OK
결과 화면 렌더링 OK	데이터가 화면 간 정상 전달
각 화면 단독 동작	전체 흐름 한 번에 통과

안티패턴 각 화면이 따로따로 동작하니까 전체도 동작할 거야 → 통합 검증 누락	통합 검증 입력 화면에서 결과 화면이 한 흐름으로 통과하는지 E2E 테스트로 증명한다
---	---

| 챕터 용어 해설

용어	정의
단위 테스트	한 함수가 올바른 값을 반환하는지 검증
통합 테스트	여러 모듈 연결 시 데이터 흐름이 정상인지
E2E 테스트	사용자가 처음부터 끝까지 시나리오 완수
시나리오 통과	화면 땀과 다른 차원의 검증. 동작의 진짜 정의

— SECTION 2

Playwright MCP와 자율 디버깅 3원칙

- 01 Playwright MCP의 사용 흐름 4단계를 외운다
- 02 자율 디버깅 3원칙을 본인 명령에 적용한다
- 03 무한 루프에 빠졌을 때 멈추는 법을 안다

PLAYWRIGHT 정의

Playwright의 정체

PLAYWRIGHT

브라우저 자동화 도구. 코드로 사람처럼 브라우저 조작. 클릭, 입력, 페이지 이동 자동화.

— 도구 정의 ★ 본 자료 핵심

Playwright MCP

PLAYWRIGHT MCP

Claude Code가 Playwright를 자연어로 조작하는 통합.

시나리오를 자연어로 설명 → 테스트 코드 생성 → 실행까지. 작업자가 테스트 코드를 직접 작성하지 않는다.

Playwright MCP 사용 흐름 4단계

단계	작업
1	PRD에서 핵심 시나리오 1개 선택
2	자연어로 시나리오 설명 → 테스트 코드 생성
3	테스트 실행과 통과 여부 확인
4	실패 시 자율 디버깅 3원칙 적용

MCP 사용 흐름 · 1단계

1단계: 시나리오 선택

사용자가 처음부터 끝까지 거치는 흐름 1개.

시드 6번 예시

"셀러가 카피 입력 화면에 상품 사진과 키워드 5개를 입력
→ '카피 생성' 버튼 클릭
→ 결과 화면에서 차별화 카피 3개 확인
→ 복사 버튼으로 카피 1개 클립보드에 복사"

— 2단계

2단계: 자연어 테스트 작성

PRD의 핵심 시나리오를 E2E 테스트로 만들어줘: "셀러가 카피 입력 화면에 상품 사진과 키워드 5개를 입력하고 '카피 생성' 버튼을 클릭하면 결과 화면에 차별화 카피 3개가 표시된다" Playwright MCP로 테스트 코드 작성과 실행까지 해 줘.

테스트 코드를 직접 작성하지 않음. 시나리오만 자연어로.

3단계

3단계: 실행과 결과 확인

[통과]

→ 다음 검증으로

[실패]

→ 자율 디버깅 루프로

두 번째 단정

자율 디버깅 3원칙:
에러 그대로 전달, 분석 먼저, 결과 검증

정의

자율 디버깅이란

AUTONOMOUS DEBUGGING

에이전트가 에러 분석 → 수정 → 재실행을 반복하는 흐름.

작업자

명령자

에이전트

디버깅 수행자

원칙 1

에러를 그대로 전달

안티패턴

- 작업자가 에러 요약
- "안 돼요"
- "에러 났어요"

올바른 운영

- 풀 메시지 그대로 복사
- 전체 스택 트레이스
- 파일 경로, 라인 번호 포함

에러 메시지가 길고 무서워 보여도 그대로. 요약하면 에이전트가 무엇이 문제인지 모름.

원칙 2

분석을 먼저, 수정은 그다음

"분석해주세요"부터

[작업자가 분석 결과 검토]

"수정해주세요"로 이어감

"고쳐주세요" 바로 → 에이전트 추측. "분석 → 수정" 순서 → 작업자가 방향 검토 가능.

원칙 3

결과를 검증

[수정 완료] ← 그대로 신뢰 X

테스트 재실행

[통과 확인] ← 작업자가 직접

"수정 완료"를 그대로 신뢰하지 않음. 통과하지 않으면 디버깅 루프 한 번 더.

표준 명령

3원칙이 적용된 표준 명령

다음 E2E 테스트 실패 메시지를 분석해주세요:

[에러 전체 메시지를 그대로 붙여넣기]

실패 원인을 코드의 어느 부분에서 찾을 수 있는지
분석한 뒤, 수정 방안을 제시하고 적용해주세요.
수정 후 테스트를 재실행해 통과 여부를 확인해주세요.

— 안티패턴 VS 3원칙

에러 보고 + 무한 루프 정리

안티패턴

"안 돼요 고쳐주세요"
"수정 끝났다고 했어요"

3원칙 적용

"분석해주세요. 풀 에러 첨부"
"재실행해 통과 확인"

무한 루프 정리 명령 (같은 자리 3회 이상 맴돌면)

이 디버깅의 어디서 막혔는지 정리해줘.
지금까지 시도한 수정 3개와 각 수정 후의 에러를
표로 보여줘.

용어 해설

용어	정의
Playwright	브라우저 자동화 도구. E2E 테스트의 표준
Playwright MCP	Claude Code가 Playwright를 자연어로 조작
자율 디버깅	에이전트가 분석·수정·재실행 반복
자율 디버깅 3원칙	에러 그대로, 분석 먼저, 결과 검증
무한 루프 정리	같은 자리 맴돌면 표로 정리해 작업자 판단

— SECTION 3

docs/prd.md 기반 시나리오 검증 루프

- 01 코드 단위 검증과 시나리오 단위 검증의 차이를 안다
- 02 scenario-verifier sub-agent의 책임을 설명한다
- 03 검증 루프의 반복 절차를 외운다

검증 단위 비교

코드 단위 vs 시나리오 단위

자료 6의 검증 vs 본 단계의 검증

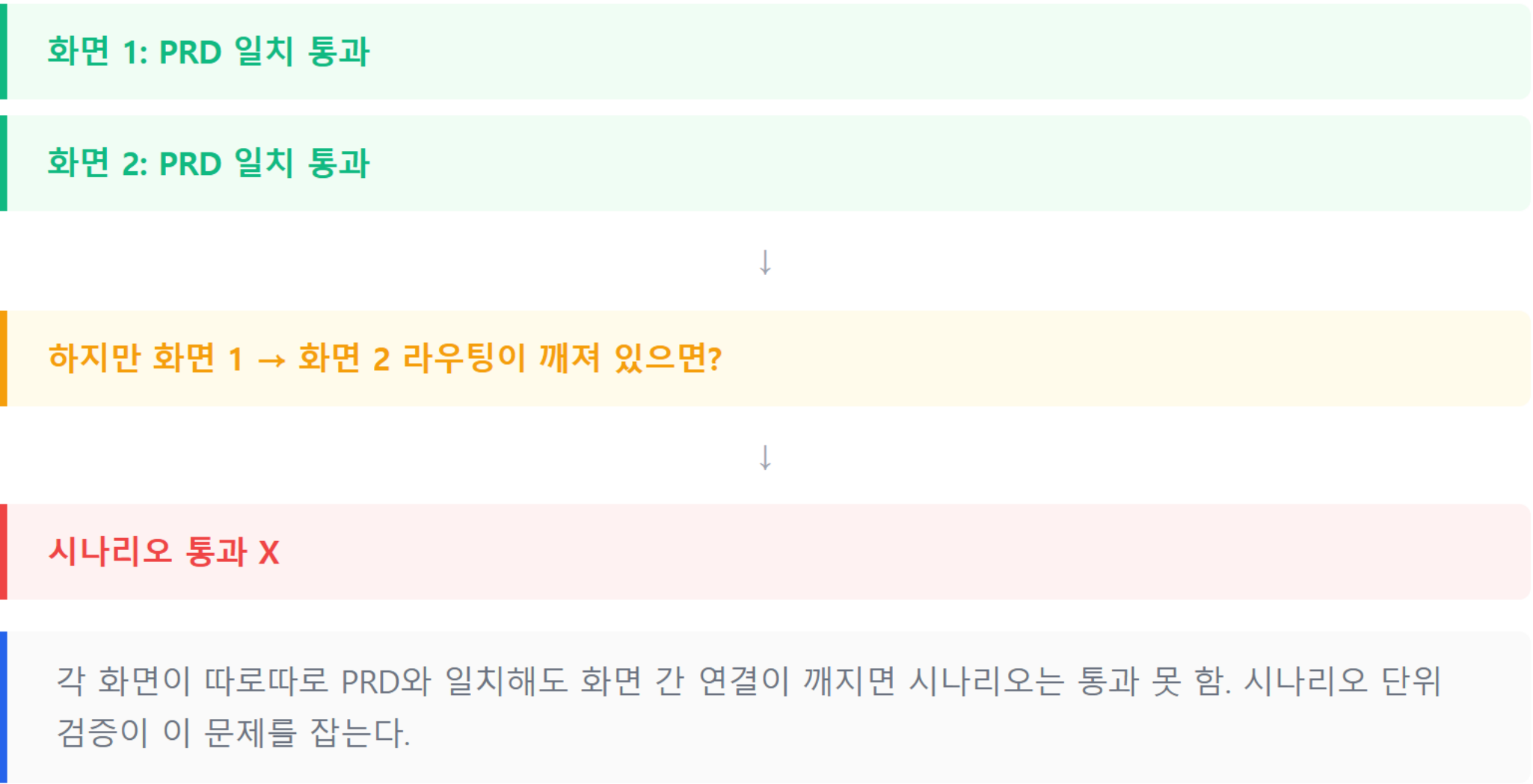
비교	코드 단위 (자료 6)	시나리오 단위 (본 단계)
시점	코드 한 개 작성 직후	전체 빌드 후
단위	한 화면 또는 한 함수	사용자 시나리오 한 묶음
잡아내는 문제	화면별 PRD 일치	화면 간 연결과 흐름
본 학습 도구	prd-reviewer 자동 (PostToolUse)	scenario-verifier 명시 호출

세 번째 단정

**PRD 일치성 검증은
코드 단위가 아닌 시나리오 단위로 한다.
그리고 검증은 루프로 진행한다**

한계 사례

코드 단위 검증의 한계



검증 도구의 진화

자료 3	Manyfast로 PRD 작성
자료 4	Manyfast → docs/prd.md export (Manyfast MCP 유료화로 제외)
자료 6	prd-reviewer가 docs/prd.md를 Read로 직접 읽고 코드 단위 검증
자료 7 (본 자료)	scenario-verifier가 docs/prd.md를 Read로 읽고 시나리오 단위 검증
본 학습 전체에서 Manyfast MCP는 PRD 작성 단계만 사용. 검증은 docs/prd.md를 직접 읽는 방식으로 일관.	

scenario-verifier sub-agent

직전 자료(자동화 학습)의 prd-reviewer와 책임 분리된 새 sub-agent. Agent 설계 6원칙(단일 책임) 적용.

자료 6 = 직전 학습(확장과 자동화) · 자료 7 = 본 학습(통합 디버깅과 E2E 테스트)

항목	prd-reviewer (직전 학습)	scenario-verifier (본 학습)
책임	코드 단위 검증	시나리오 단위 검증
호출 시점	코드 수정 직후 (자동)	빌드 완료 후 (명시)
입력	수정된 파일 1개	시나리오 1개 + 전체 코드베이스
출력	일치/불일치/검토 외 3카테고리	시나리오 통과/막힘 지점/수정 제안

scenario-verifier 표준 구조

Agent 설계 6원칙 모두 적용 + 책임 분리

파일 경로: `.claude/agents/scenario-verifier.md`

```
---
name: scenario-verifier
description: 빌드 완료 후 호출.
  docs/prd.md의 핵심 시나리오 1개와 전체 코드를 비교해
  시나리오가 처음부터 끝까지 통과 가능한지 검증합니다.
tools: Read, Grep, Glob
model: sonnet
---
```

시나리오 단위 PRD 검증 전문가의 역할.

****입력****: 검증할 시나리오 설명 1개
(예: "셀러가 카피 입력 → 결과 확인")

****작업 절차****:

- docs/prd.md를 Read로 읽고 시나리오 흐름 확인
- Grep/Glob으로 시나리오 관련 파일 식별
- 각 단계가 코드로 구현됐는지 추적
- 화면 간 라우팅과 데이터 전달 점검
- 막힘 지점이 있으면 구체적 위치 보고

****출력 형식**** (3단 구조):

- 시나리오 통과 여부: 통과 / 부분 통과 / 미통과
- 막힘 지점: 어느 단계가 어디서 막히는지 (파일·줄)
- 수정 제안: 무엇을 추가/수정해야 하는지

****주의****:

- 수정 권한 없음 (tools에 Write/Edit 없음)
- 추측 금지. docs/prd.md와 실제 코드만 보고 판단
- 보고서는 한국어

Agent 설계 6원칙 적용

원칙	적용 위치
1. 단일 책임	"시나리오 단위 PRD 검증" 한 줄
2. 최소 권한	Read, Grep, Glob만 (Write 없음)
3. 결정적 출력	통과/막힘/제안 3단 고정
4. 책임 분리	"수정 권한 없음" 명시
5. 호출 시점	"빌드 완료 후 호출"
6. 입출력 계약	시나리오 입력 → 3단 출력

시나리오 검증 루프

본 단계의 검증은 단발 호출이 아니라 루프이다.

[1] scenario-verifier 호출 (시나리오 입력)

[2] 보고서 수신: 통과 / 부분 통과 / 미통과

[3] 작업자가 보고서 검증 (막힘 지점 진짜? 수정 제안 적절?)

[4-a] 통과 → 다음 단계 (배포)

[4-b] 미통과 → 코드 수정

[5] 코드 수정 후 다시 [1]로

[루프 반복: 통과까지]

시나리오 검증 명령

자료 6 PRD-REVIEWER (코드 단위)

@prd-reviewer src/components/InputForm.tsx를 검토해줘

자료 7 SCENARIO-VERIFIER (시나리오 단위)

@scenario-verifier 다음 시나리오를 검증해줘:

"셀러가 카피 입력 화면에 상품 사진과
키워드 5개를 입력하고 '카피 생성' 버튼을 클릭하면
결과 화면에 차별화 카피 3개가 표시된다"

docs/prd.md를 읽고 현재 코드가 이 시나리오를
처음부터 끝까지 동작시킬 수 있는지 점검해줘.

시드 6번 첫 검증 후 보고서

[시나리오 통과 여부]
부분 통과

[막힘 지점]

- 단계 1 (입력): src/components/InputForm.tsx에서 정상 동작
- 단계 2 (라우팅): src/app/page.tsx의 onSubmit이 '/result'로 이동하지만 입력값을 query로 안 넘김
- 단계 3 (결과 표시): src/app/result/page.tsx가 입력값을 받지 못해 빈 화면 표시

[수정 제안]

1. page.tsx의 onSubmit에 router.push 시 query 파라미터 추가
2. result/page.tsx에서 useSearchParams로 입력값 수신
3. 수신한 입력값을 OpenAI API 호출에 전달

sub-agent 보고서도 AI 산출물

검증 우선주의의 일관 적용 — 자동 신뢰 금지

점검 항목	방법
막힘 지점 1-2개 직접 확인	보고서가 가리킨 파일줄 열어보기
수정 제안 적절성 점검	본인이 PRD와 대조
자동 신뢰 금지	환각 가능성 인식

안티패턴 vs 검증 루프

안티패턴

'보고서가 통과라고 했으니 다음 단계'
→ 검증 우선주의 위반

'한 번 호출하고 결과 그대로 수용'
→ 루프 누락

올바른 운영

보고서의 막힘 지점 1-2개 직접 확인. 수정 후 다시 호출. 통과까지 반복.

용어 해설

용어	정의
시나리오 단위 검증	코드 한 줄이 아닌 사용자 흐름 한 묶음으로 검증
scenario-verifier	시나리오 단위 검증 전문 sub-agent
검증 루프	검증 → 수정 → 재검증을 통과까지 반복
PRD 일치성	만든 것이 PRD의 의도대로 동작하는가
두 갈래 결과	결과는 항상 통과 또는 미통과 (부분 통과 포함)